CHEAT SHEET

# RAG System Workflow

Getting Started • Pipeline • Storage • Components • Advanced Retrieval • Safety

Workflow

Pipeline

# 1. Getting Started

Basic steps to start building a RAG pipeline.

**Core Goal**

Connect **knowledge** → **retrieval** → **reasoning**

```
pip install langchain chromadb sentence-transformers
```

**Steps:**

- Collect documents
- Preprocess data
- Generate embeddings
- Store in vector DB
- Retrieve relevant chunks
- Send to LLM

# 2. Understanding RAG Pipeline

RAG = **Retrieval Augmented Generation**

**User Query**

↓

**Query Embedding & Vector Search**

↓

**Retrieve Top K Documents**

↓

**Context Injection & LLM Generation**

# 3. Knowledge Storage Hierarchy

Where knowledge lives in a RAG system.

### Data Sources

PDFs, Websites, Databases, APIs

### Processing Layer

Chunking, Cleaning, Metadata

### Vector Layer

Embeddings, Vector DB

**Example Vector DBs**

Chroma, Pinecone, Weaviate, Qdrant

# 4. RAG Best Practices

- Chunk size: **300–800 tokens**
- Use **semantic chunking**
- Add **metadata filtering**
- Re-rank retrieved documents
- Avoid sending too many chunks to LLM
- Track retrieval scores

**Remember**

Good RAG is **retrieval engineering**, not just prompting.

# 5. Project File Structure

Example RAG project layout.

```
rag-system/
├─ data/
│  ├─ raw_docs/
│  └─ processed_docs/
│
├─ ingestion/
│  ├─ chunking.py
│  └─ embeddings.py
│
├─ retrieval/
│  ├─ retriever.py
│  └─ reranker.py
│
├─ generation/
│  ├─ prompts.py
│  └─ llm.py
│
├─ evaluation/
│  └─ rag_eval.py
│
└─ app.py
```

# 6. Core Components of RAG

Essential building blocks.

### Chunking

Split docs into segments.

- → Fixed chunking
- → Recursive chunking
- → Semantic chunking

### Embeddings

Convert text to vectors.

- → OpenAI
- → Gemini
- → BGE, E5

### Vector Database

Stores embeddings for retrieval. Functions: similarity search, filtering, hybrid search.

# 7. RAG Ideas for AI Engineers

Common systems you can build.

- **Chat with PDFs**
- **Company knowledge assistant**
- **Customer support bot**
- **Research assistant**
- **Code documentation assistant**
- **Personal knowledge base**

# 8. Advanced Retrieval Techniques

Improve search quality.

## Hybrid Search

```
Vector Search + Keyword Search
```

## Re-ranking

```
Retrieve 20 docs
Re-rank top 5
Send to LLM
```

## Query Expansion

```
User Query → Multiple Reformulated Queries
```

## Multi-Hop Retrieval

Used for complex reasoning questions where information is spread across multiple documents.

# 9. Safety & Guardrails

Prevent failures in RAG systems.

### Permissions & Safety

- Limit document sources
- Add prompt safety filters
- Prevent data leakage
- Use source citations
- Track hallucination rate

## Example prompt rule:

```
Answer ONLY using provided context.
If information not found, say "Not in knowledge base".
```

# 10. RAG Architecture Layers

Typical production architecture.

**Layer 1 – Data Sources**

PDFs, APIs, Databases

**Layer 2 – Ingestion Pipeline**

Cleaning, Chunking, Embeddings

**Layer 3 – Retrieval Engine**

Vector DB, Re-ranking

**Layer 4 – Generation**

LLM + Prompt + Context

# 11. Daily RAG Workflow

Typical development loop.

**Add new documents & Run ingestion**

↓

**Test queries → Check retrieval quality**

↓

**Tune chunk size & Tune top_k**

↓

**Add re-ranking / Improve prompts**

**Best habit**

**Evaluate retrieval before blaming the LLM.**

# 12. Quick Reference

- **Chunk Size:** 500 tokens
- **Top-K Retrieval:** 3 – 5 docs
- **Vector DB Query:** similarity_search(query, k=5)
- **Embedding Model:** text-embedding-3-large, bge-large

**Start Building!**

Build your first RAG pipeline today.